

README

Helix Constitution-Powered SpecKit-Superpowers Bridge: Implementation Documentation

Revision: 1 **Last modified:** 2026-07-24T12:00:00Z **Description:** Master index and architecture overview for the Helix Constitution-powered SpecKit-Superpowers Bridge implementation. **Maintainer:** HelixDevelopment **Scope:** `constitution/docs/research/extensions/speckit_superpowers/implementation/`

Table of Contents

1. [Executive Summary](#)

2. [System Architecture](#)

- [Architecture Diagram](#)
- [Layer 1 — Developer Workstation](#)
- [Layer 2 — Spec-Kit Core \(Governance Layer\)](#)
- [Layer 3 — SuperSpec Bridge \(Orchestration Layer\)](#)
- [Layer 4 — SuperB Extension \(Discipline Enforcement\)](#)
- [Layer 5 — SuperBridge MCP \(Execution Layer\)](#)
- [Layer 6 — Helix LLM \(Inference Layer\)](#)
- [Layer 7 — Distributed Host Cluster](#)

3. [Data Flow](#)

- [Feature Lifecycle Through the Layers](#)
- [Nano-Task Decomposition Flow](#)

- 4. [Document Index](#)
 - 5. [Quick Start](#)
 - 6. [Key Design Decisions](#)
 - 7. [Constitutional Mandates Applied](#)
 - 8. [Directory Structure](#)
 - 9. [References](#)
-

Executive Summary

This documentation defines the **Helix Constitution–Powered SpecKit–Superpowers Bridge** — a unified, multi-layer development system that integrates the full SDD (Specification-Driven Development) lifecycle of [Spec-Kit](#) with the agent-skill ecosystem of [Superpowers](#), orchestrated through the [SuperSpec](#) and [SuperB](#) bridge extensions, with distributed inference powered by [Helix LLM](#) backed by a `llama.cpp` RPC cluster.

Problem It Solves

Modern software development with LLM-assisted agents suffers from four structural problems:

Problem	Manifestation	Root Cause
Context fragmentation	Each tool (specifier, planner, coder, reviewer) runs in its own silo; no shared understanding of what is being built	No canonical bridge between specification artifacts and execution tools
Discipline drift	Specifications written but never consulted during implementation; plans that do not reflect reality; "fix it later" culture	No mechanical enforcement that the spec <i>governs</i> the work
Inference latency & cost	Centralized LLM APIs introduce round-trip latency, rate limits, and per-token billing that blocks autonomous-loops	No local-first distributed inference pipeline
Anti-bluff gap	Tests pass but features do not work; configuration-only PASS; metadata-only PASS; grep-without-runtime PASS	No §11.4 positive-evidence mandate wired into the development lifecycle

This bridge closes all four gaps by composing Spec-Kit's `specify → plan → tasks → implement` lifecycle with Superpowers' `brainstorm / debug / review / test` skills under a single Constitution-governed execution framework, all powered by local distributed inference that keeps the autonomous loop running without API rate-limit interruptions.

What This Documentation Covers

1. The **7-layer system architecture** with each layer's responsibilities, technologies, and contractual interfaces.
2. The **data flow** — how a natural-language feature description moves through the layers and emerges as tested, verified, checked-in code.
3. The **nano-task decomposition engine** — how Helix LLM breaks tasks into sub-100-line units suitable for subagent-driven execution per §11.4.70.
4. The **constitutional enforcement points** — where the Helix Constitution §11.4 anti-bluff covenant, TDD-fix-discipline, mechanical enforcement, and reporting-directive system plug into the bridge.

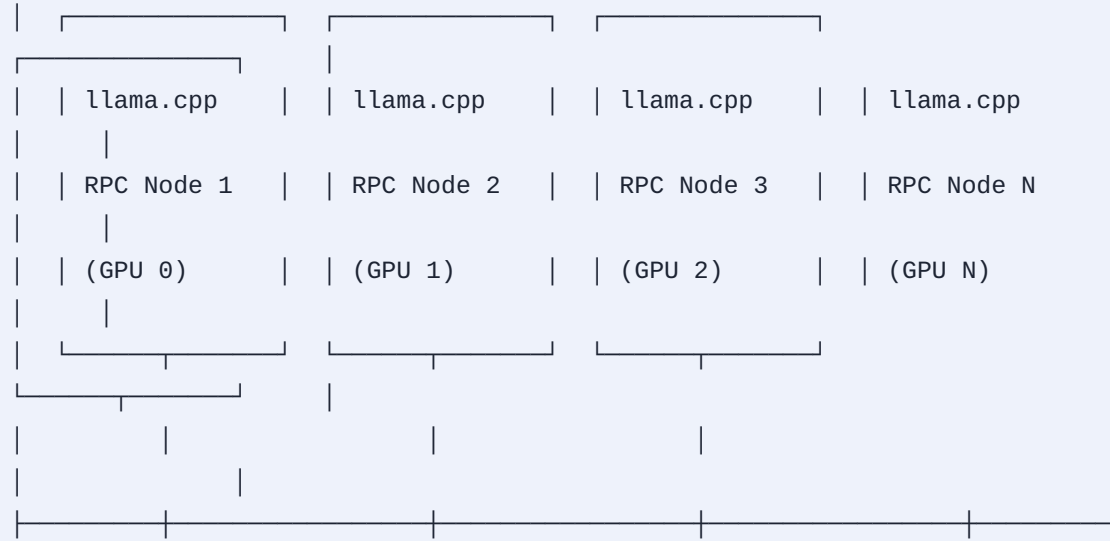
5. The **onboarding path** — how any consuming project (e.g., ATMOSphere) integrates the Constitution submodule, wires its CLI-agent context carriers, installs the bridge extensions, and goes from zero to full autonomous-loop development.
-

System Architecture

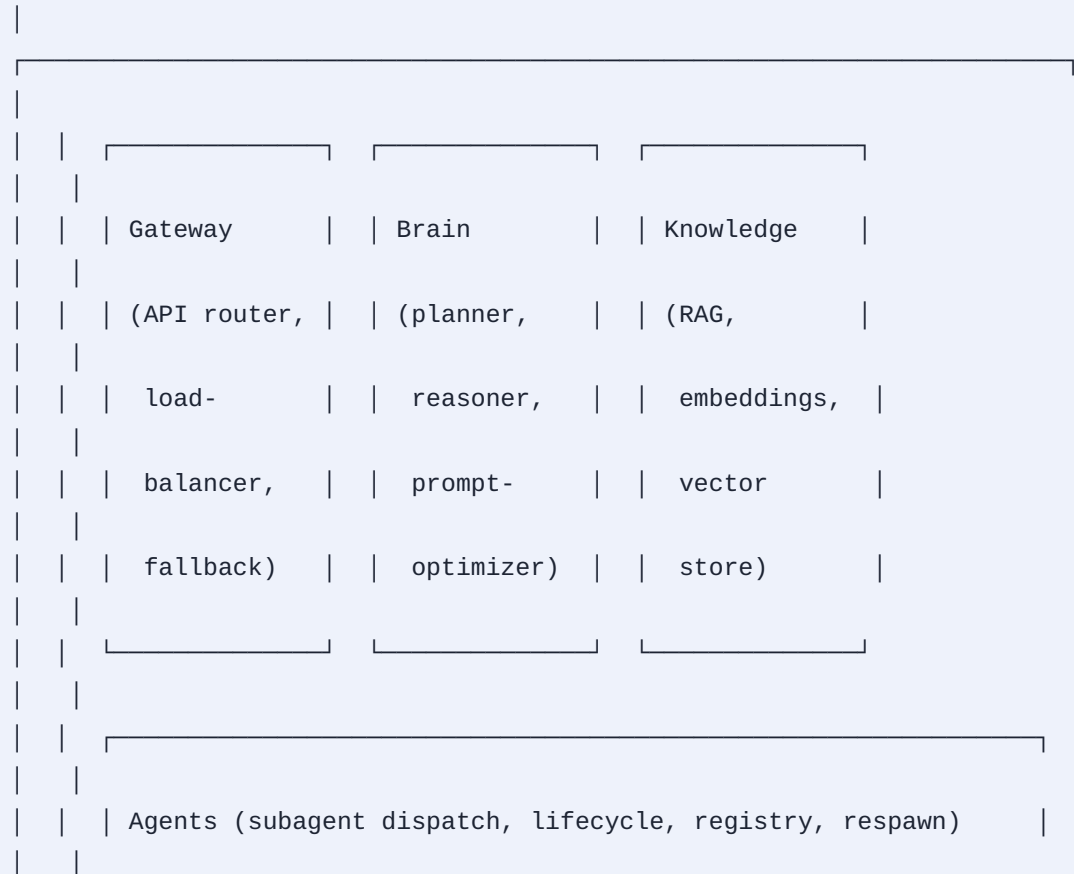
Architecture Diagram

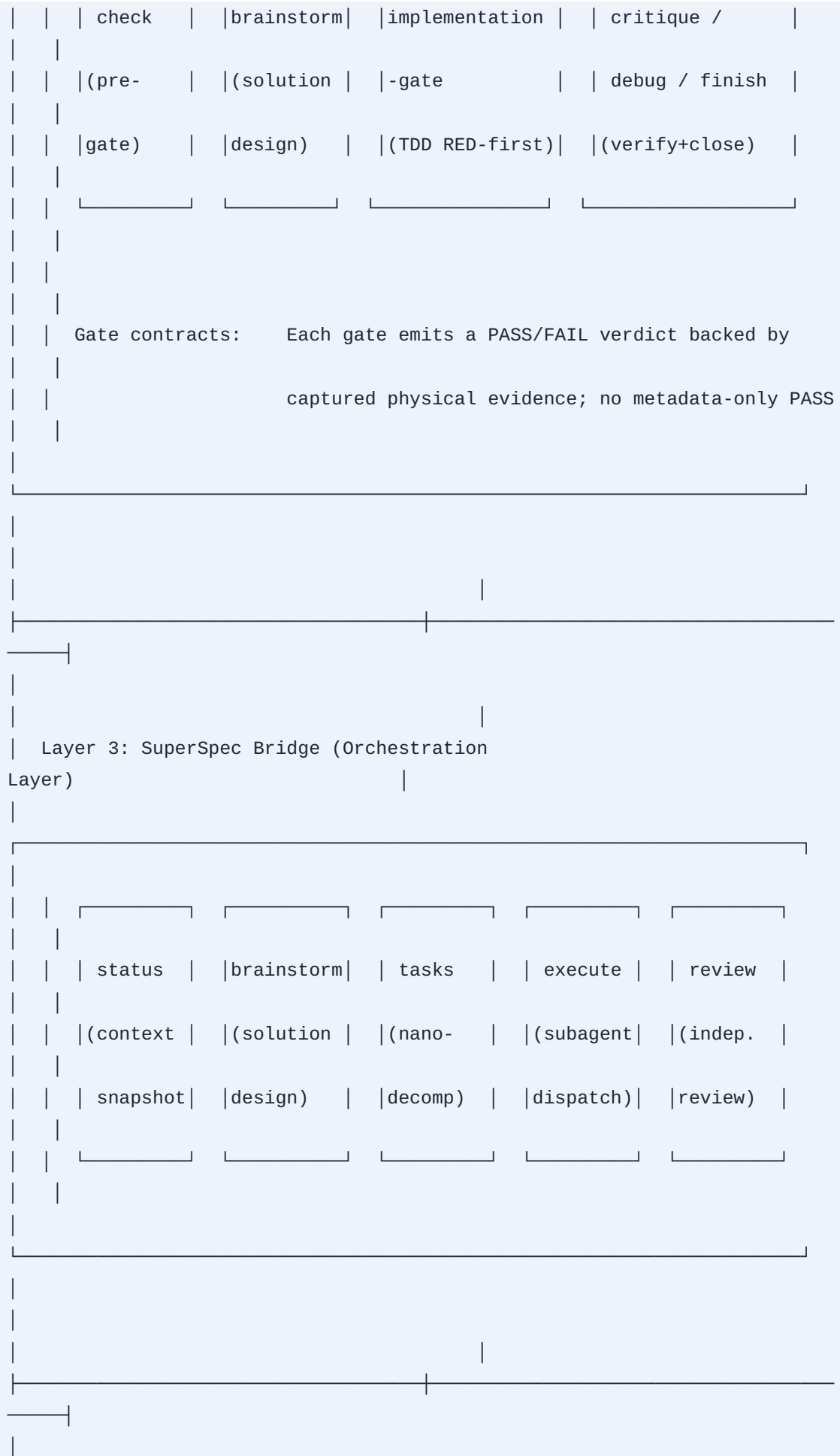
Layer 7: Distributed Host

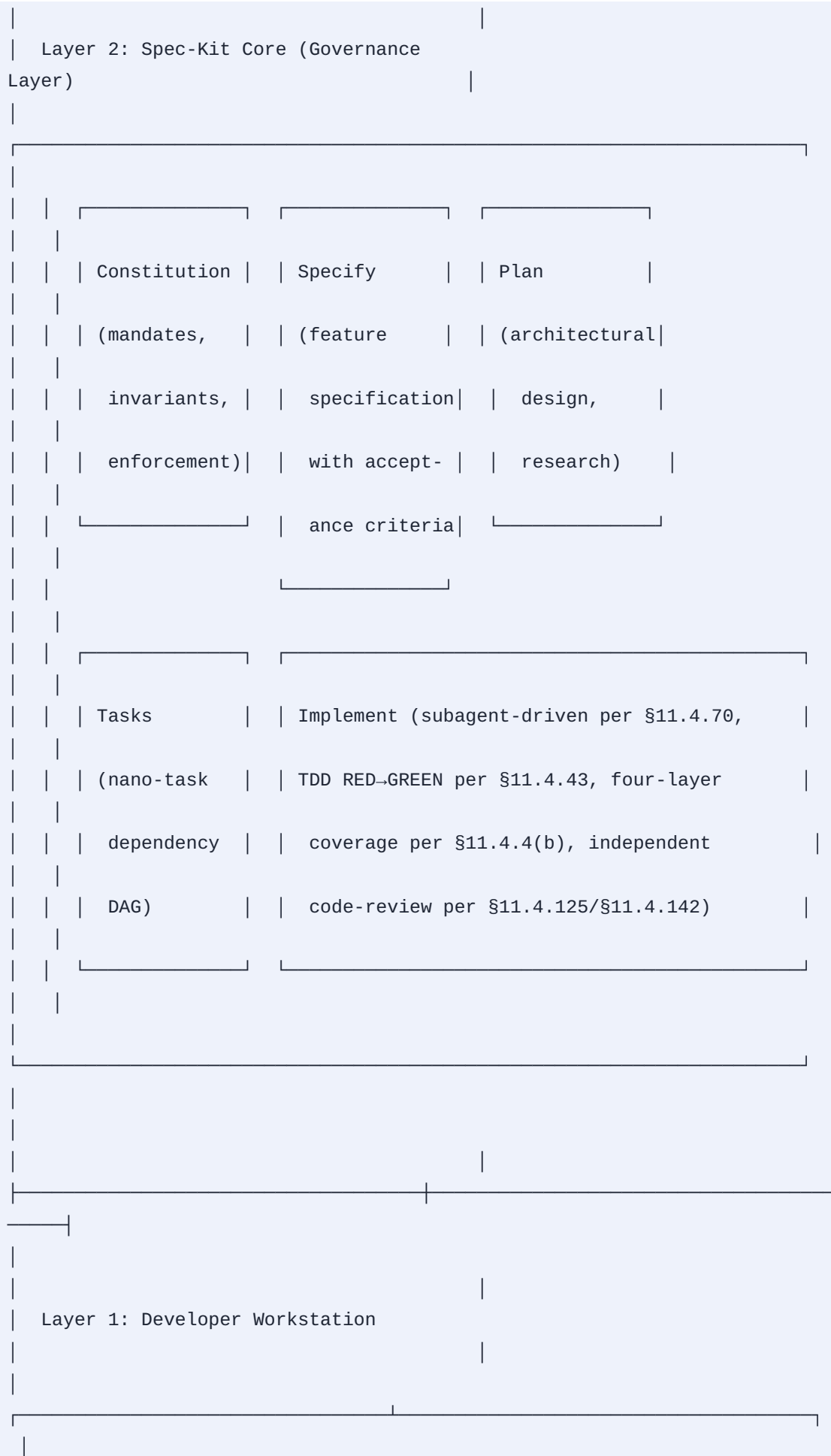
Cluster

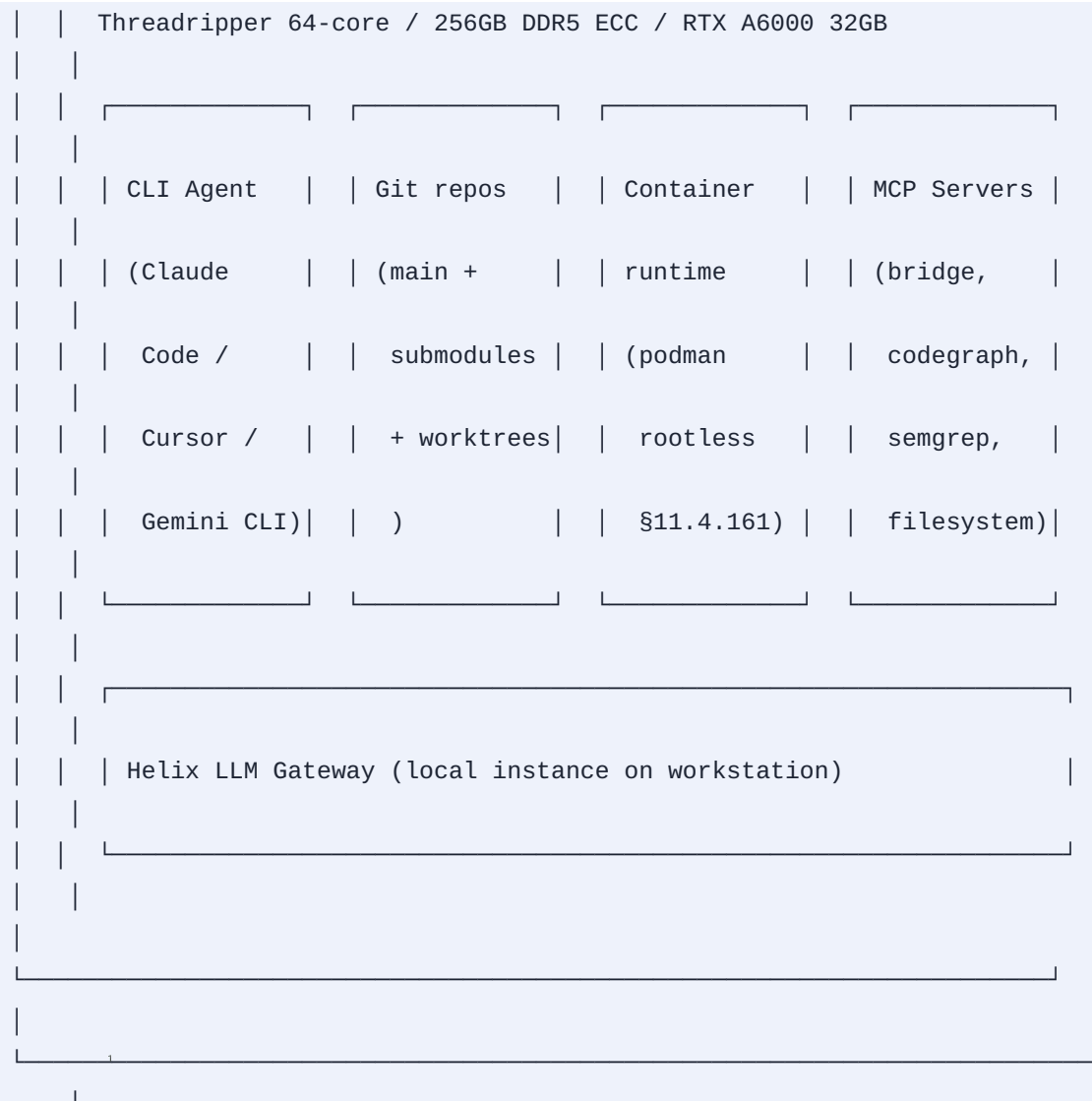


Layer 6: Helix LLM (Inference Layer)









Layer – Developer Workstation

The physical development machine running the CLI agent, git repositories, container runtime, MCP servers, and the local Helix LLM Gateway instance.

Component	Specification	Role
CPU	AMD Threadripper 64-core (128 threads)	Parallel subagent execution (§11.4.70/.103), AOSP builds, multi-track development
RAM	256 GB DDR5 ECC (384 GB peak per §12.6 ceiling)	Bounded by §12.6 60% memory budget; <code>bounded_run</code> enforced
GPU	NVIDIA RTX A6000 32 GB VRAM	Local inference for Helix LLM Gateway; offloads smaller models
Storage	NVMe RAID, ≥4 TB	btrfs CoW for feature work-streams per §11.4.167
Container Runtime	Podman rootless (§11.4.161)	AOSP 42 GB containerised builds per §12.9
MCP Servers	CodeGraph, SuperBridge, filesystem	Tool-accessible subprocesses providing structured agent capabilities
CLI Agent	Claude Code / Cursor / Gemini CLI / Qwen Code	The human-facing agent conducting the autonomous loop (§11.4.126)
Git Layout 2	Multi-track with own <code>.git</code> per track (§11.4.179)	Isolated concurrent development streams

Layer — Spec-Kit Core (Governance Layer)

The canonical [Spec-Kit](#) SDD toolkit, extended by the Helix Constitution. This is the **source of truth** for what is being built and how.

Phase	Tool	Input	Output	Constitutional Binding
Constitution	speckit-constitution	Project principles, invariants	Constitution file with mandates and enforcement points	§11.4 (anti-bluff covenant), §11.4.6 (no-guessing), §11.4.10 (credentials), §11.4.75 (mechanical enforcement), §11.4.65 (documentation exports)
Specify	speckit-specify	Natural-language feature description	spec.md with acceptance criteria, edge cases, topology constraints	§11.4.91 (≥6-word/≥40-char clear meaning), §11.4.146 (reproduce-first test workflow)
Plan	speckit-plan	spec.md + constitution.md	plan.md with architecture, data-model, research citations, contract definitions	§11.4.8 (deep-web-research citation), §11.4.99 (latest authoritative sources), §11.4.150 (multi-angle research pass)
Tasks	speckit-tasks	spec.md + plan.md	tasks.md with dependency-ordered nano-task DAG	§11.4.70 (subagent-driven default), §11.4.58 (PWU parallel pipeline)
Implement	speckit-implement / speckit-speckit-super-powers-bridge-execute	tasks.md → bridge dispatch	Checked-in code with four-layer coverage	§11.4.43 (TDD RED → GREEN), §11.4.4(b) (four-layer coverage), §11.4.125/ §11.4.142 (code review)

Governance enforcement — The Constitution file produced by `speckit-constitution` contains:

- **Invariants** — assertions that must hold true at every gate (e.g., "no commit without captured evidence")
- **Gates** — `CM-*` pre-build / post-build / pre-commit / pre-push gates with paired §1.1 meta-test mutations
- **Propagation directives** — §11.4.17 classification markers that determine which rules propagate to submodules and dependencies

Layer — SuperSpec Bridge (Orchestration Layer)

The [SuperSpec](#) bridge sits between Spec-Kit's governance artifacts and the execution layer. It translates `spec.md` / `plan.md` / `tasks.md` into structured workflows that can be dispatched to subagents.

Command	Purpose	Input	Output
<code>superspec status</code>	Capture current project context snapshot	All <code>.md</code> artifacts + git state	Structured context JSON fed to every sub-agent dispatch
<code>superspec brainstorm</code>	Generate solution approaches	<code>spec.md</code> + <code>plan.md</code> + constitution	Ranked solution alternatives with trade-offs
<code>superspec tasks</code>	Nano-task decomposition DAG	<code>spec.md</code> + <code>plan.md</code> + <code>tasks.md</code>	Sub-100-line tasks with file-scope manifests per §11.4.58
<code>superspec execute</code>	Subagent dispatch loop	Nano-task DAG + context snapshot	Checked-in code with verdict files per task
<code>superspec review</code>	Independent code-review dispatch	Change diff + context snapshot	Review findings + GO/NO-GO verdict per §11.4.142

Nano-task contract — Each nano-task MUST:

- Be ≤ 100 lines of net-new / changed code (breaker rule: a task exceeding this splits)
- Declare a file-scope manifest (exactly which files it touches, per §11.4.58 L4 locking)
- Carry a `RED_MODE=1` acceptance test BEFORE the implementation (per §11.4.115/§11.4.146)
- Emit a machine-written verdict file (per §11.4.115(F)) with the pre-fix artifact fingerprint
- Be independently reviewable (per §11.4.142 — a nano-task is the atomic review unit)

Layer — SuperB Extension (Discipline Enforcement)

The [SuperB](#) extension enforces Spec-Kit discipline mechanically — no implementation proceeds past any gate that fails its §11.4.107(10) golden-good/golden-bad self-validation.

Gate	When It Fires	What It Checks	Pass Condition
superb check	Pre-specify, pre-plan, pre- implement	Constitution invariants, spec completeness, plan consist- ency	All CM-* gates GREEN
superb brain- storm	After speck- it-plan , be- fore speckit- tasks	Solution design covers all ac- ceptance criteria, cites re- search, enumerates alternat- ives	Zero unexamined acceptance criteria
superb imple- menta- tion- gate	Before superspec execute dis- patches any subagent	RED test exists per nano-task, file-scope manifests declared, pre-build gates wired	All RED tests parse with sh -n / bash -n per §11.4.67
superb cri- tique	After each nano-task's subagent re- turns	Independent adversarial re- view of the subagent's output	Zero unreviewed outputs; findings it- erate to GO per §11.4.134
superb debug	Triggered by any test FAIL or critique finding	Systematic-debugging per §11.4.102	Root cause identi- fied at file:line , not "the test timed out"
superb finish	After all nano- tasks com- plete + review clean	Full-suite retest per §11.4.40, meta-test mutation sweep per §1.1, Issues.md → Fixed.md migration per §11.4.19	Zero FAIL, zero WARN, zero un- covered verdicts

Discipline enforcement is mechanical, not advisory — per §11.4.205, every gate is a `commit_all.sh` -wrapper refusal, not a prose suggestion. The gate's paired §1.1 mutation **MUST FAIL** the gate; a gate whose mutation passes is a bluff gate and a release blocker.

Layer – SuperBridge MCP (Execution Layer)

An MCP (Model Context Protocol) server that exposes the Superpowers skill suite as tool-callable endpoints consumable by any MCP-compatible CLI agent.

Tool	Superpowers Skill	Contract
<code>super-bridge_brainstorm</code>	<code>super-powers:brainstorming</code>	Input: problem statement + constraints. Output: ranked alternatives with evidence per §11.4.69.
<code>superbridge_debug</code>	<code>superpowers:systematic-debugging</code>	Input: failure evidence (logcat, tombstone, crash dump). Output: <code>file:line</code> root cause + blast radius.
<code>super-bridge_review</code>	<code>superpowers:executing-code-review</code>	Input: diff + context. Output: findings classified (BLOCKING/WARNING/NIT) per §11.4.134.
<code>super-bridge_test_gen</code>	<code>superpowers:writing-tests</code>	Input: function signatures + spec. Output: TDD RED test with four-layer coverage.
<code>super-bridge_red_green</code>	<code>superpowers:executing-testing-tdd</code>	Input: RED test. Output: GREEN implementation + verdict file per §11.4.115(F).
<code>super-bridge_analyze</code>	<code>superpowers:systematic-debugging</code> (impact)	Input: change diff. Output: §11.4.145 eight-angle impact research report.
<code>super-bridge_refactor</code>	Writing clean / refactoring code	Input: target file + goal. Output: refactored code with preserved behaviour proven by tests.
<code>super-bridge_document</code>	<code>superpowers:writing-documentation</code>	Input: code + context. Output: §11.4.44 revision-headed, §11.4.65 four-format-exported doc.

Tool	Superpowers Skill	Contract
<code>super-bridge_search</code>	Web research per §11.4.8 / §11.4.99 / §11.4.150	Input: research question + angle set. Output: cited findings with URLs + access dates.

MCP server requirements:

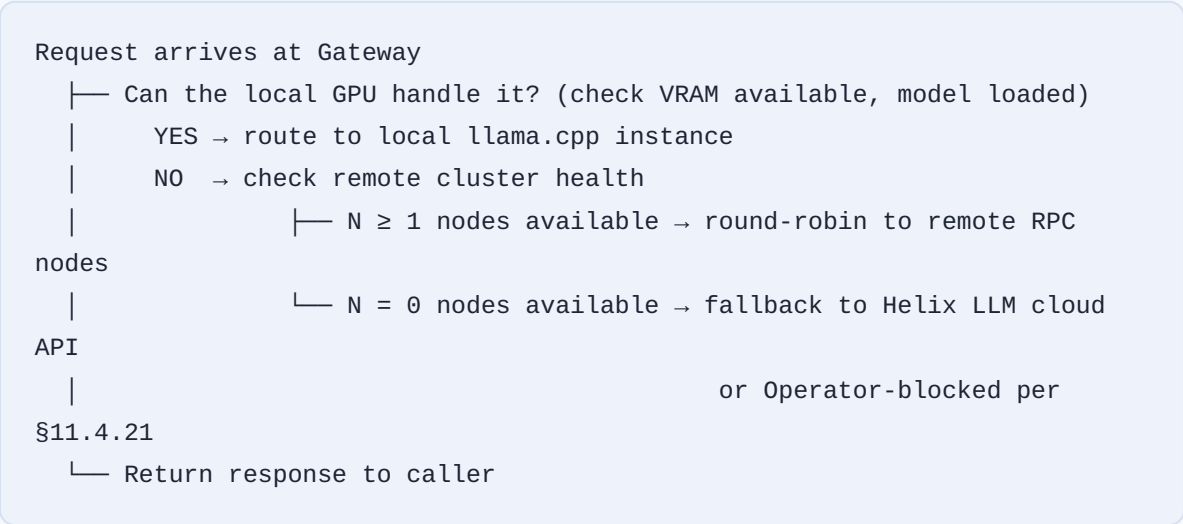
- Runs rootless per §11.4.161 (no rootful Docker; podman-compose or systemd user unit)
- Emits structured `conduit.events.jsonl` per §11.4.116 (append-only event stream)
- All verdicts carry `evidence_path` per §11.4.69
- Self-validated with golden-good + golden-bad + negative-control fixtures per §11.4.107(10)

Layer – Helix LLM (Inference Layer)

The [Helix LLM](#) distributed inference system provides **local-first, zero-cost-per-token** inference powering the entire bridge pipeline.

Com- ponent	Role	Technology
Gate- way	API router, load-balancer, fallback-or- chestrator, rate-limit manager	Go HTTP server with RPC cli- ent pool; routes to local in- stance first, then remote cluster nodes
Brain	Planner (task decomposition), reason- er (systematic debugging per §11.4.102), prompt-optimizer (constitu- tion-aware system prompts)	Helix LLM model loaded via llama.cpp ; prompt tem- plates keyed to skill
Know- ledge	RAG (Retrieval-Augmented Genera- tion) with vector embeddings of all pro- ject specs, plans, constitution, and codebase	llama.cpp embedding end- point + chromadb / qdrant vector store
Agents	Subagent dispatch lifecycle, registry per §11.4.147, respawn-on-crash per §11.4.147(b), verdict tracking per §11.4.116	Go agent orchestration calling the Gateway

Gateway routing logic:



Models supported (2026-07-24):

- **Fable** (deepseek-v4-pro) — architecture design, code review (§11.4.209), merge-conflict resolution (§11.4.211)

- **Opus** (`claude-opus-4-20250514`) — fallback for Fable-only tasks, systematic debugging
- **Helix-local** (custom fine-tune on RK3588/ATMOSphere corpus) — nano-task implementation, test generation, documentation

Layer – Distributed Host Cluster

A pool of machines running `llama.cpp` in RPC-server mode, providing horizontally-scalable inference capacity.

Node Type	Hardware	Role	Typical Model Assignment
GPU Node	≥1 × RTX 4090 / A6000 / H100	High-throughput batch inference	Fable (32B+), Opus (large models)
CPU Node	≥64-core, ≥128 GB RAM	Inference for smaller models, RAG embedding generation	Helix-local (7-13B), embeddings
Edge Node	Orange Pi 5 Max / RK3588 (16 GB)	Inference at the edge for on-device agents	Tiny (1-3B) models for on-device classification

RPC configuration (per node `llama-server` invocation):

```
# On each cluster node:
llama-server \
  --model /models/deepseek-v4-pro-Q4_K_M.gguf \
  --host 0.0.0.0 \
  --port 8080 \
  --n-gpu-layers 40 \
  --ctx-size 32768 \
  --parallel 4 \
  --cont-batching \
  --mlock
```

Health monitoring:

- Per-node heartbeat at 5 s interval (via `/health` endpoint)

- Gateway auto-evicts unresponsive nodes after 3 consecutive missed heartbeats
 - Re-admission requires 5 consecutive successful heartbeats (hysteresis prevents flapping)
 - All node state changes logged to `conduit.events.jsonl` per §11.4.116
-

Data Flow

Feature Lifecycle Through the Layers

Operator prompt: "Add multi-channel audio passthrough for Dolby TrueHD"



L7: Helix LLM Brain classifies intent → "Feature", enriches with RAG context (existing audio HAL docs, ALSA config, ES8388 codec spec, HDMI EDID passthrough constraints)

enriched prompt + context



L2: speckit-specify generates spec.md:
- Feature: Dolby TrueHD passthrough over HDMI
- Acceptance criteria: Arvus AVR reports Codec-In-Use == "Dolby TrueHD" when TrueHD stream is played
- Edge cases: fallback to PCM on non-TrueHD sink, channel count mismatch, sample-rate negotiation
- Topology constraints: D1 (HDMI sink present), D2 (no sink)

spec.md



L2: speckit-plan generates plan.md:
- Research: HDMI 2.1 audio infoframe spec, CEA-861, ALSA IEC958 passthrough driver implementation, Rockchip audio HAL codec_config path
- Architecture: modify audio_hw.c out_set_parameters, add TrueHD codec profile to audio_policy_config, add Arvus sink-probe assertion to test harness
- Contract: audio_hw.c:audio_out_set_parameters() must accept AUDIO_FORMAT_TRUEHD and set IEC958 non-audio bit

plan.md



L2: speckit-tasks generates tasks.md:
T1: [RED] test_audio_truehd_passthrough_red.sh – captures pre-fix Arvus state (Codec-In-Use != TrueHD)
T2: audio_hw.c: add TrueHD format to supported codecs
T3: audio_policy_config_hifi.xml: register TrueHD devicePort
T4: Arvus probe assertion in test harness
T5: [GREEN] T1 flipped to RED_MODE=0 – captures post-fix Arvus state (Codec-In-Use == TrueHD)
Dependency DAG: T1 → T2, T3 → T4 → T5

| tasks.md



| L3+L4+L5: SuperSpec + SuperB + SuperBridge execute loop:

| superb check → superspec status → superspec tasks (nano) →
| → superb implementation-gate (verify RED tests) →
| → superspec execute (dispatch nano-subagents per task) →
| → superb critique (review each subagent output) →
| → superb debug (fix any findings) →
| → superb check (re-verify) → loop until GO

| Each nano-task subagent:

- | 1. Reads context snapshot (superspec status)
- | 2. Reads spec + plan + tasks
- | 3. Reads existing code via CodeGraph MCP
- | 4. Writes RED test first (§11.4.43 step 1)
- | 5. Writes implementation (§11.4.43 step 3)
- | 6. Runs RED test → GREEN (§11.4.43 step 4)
- | 7. Writes verdict file + evidence path (§11.4.115(F))
- | 8. Submits for review

| completed nano-tasks + verdicts



| L2: speckit-converge assesses remaining gaps, appends to
| tasks.md any unbuilt work



| Final gate (§11.4.40 full-suite):

- | - pre_build_verification.sh → ALL GREEN
- | - post_build_verification.sh → ALL GREEN
- | - On-device (D1+D2) test_all_fixes.sh → ALL GREEN
- | - meta_test_false_positive_proof.sh → ALL MUTATIONS FAIL
- | - HelixQA Challenge sweep → ALL GREEN
- | - Issues.md → Fixed.md migration → COMMITTED
- | - Tag created + pushed to all remotes + submodules

Nano-Task Decomposition Flow

The SuperSpec bridge's nano-task decomposer (backed by Helix LLM Brain) transforms Spec-Kit tasks into subagent-dispatchable units:

Input: tasks.md task "T2: audio_hw.c: add TrueHD format to supported codecs"

Step 1 – File-scope partition (§11.4.58 L4):

- └─ NT2a: audio_hw.c: add AUDIO_FORMAT_TRUEHD to format enum (7 lines)
- └─ NT2b: audio_hw.c: add TrueHD case to out_set_parameters() (23 lines)
- └─ NT2c: audio_hw.c: add TrueHD pcm_config entry (12 lines)

Step 2 – Dependency resolution:

- └─ NT2a (no deps) → dispatchable immediately
- └─ NT2b (depends on NT2a – needs the enum value) → queued after NT2a
- └─ NT2c (depends on NT2a + NT2b) → queued after NT2b

Step 3 – Parallel dispatch (when deps permit):

Queue: [NT2a]
Dispatch NT2a → subagent-1
NT2a completes → enqueue NT2b
Dispatch NT2b → subagent-2
NT2b completes → enqueue NT2c
Dispatch NT2c → subagent-3
NT2c completes → done

Step 4 – Per-nano-task verdict:

NT2a/verdict.json: {"id":"NT2a","status":"PASS","red-exit":1,"green-exit":0,
"artifact-fingerprint":"a3f8...", "evidence":"qa-results/.../NT2a/"}

Step 5 – Merge gate (§11.4.41):

All nano-tasks for T2 complete → merge subagent outputs → superb critique →
→ T2 complete → ready for T3 dispatch

Nano-task size rule (mechanical): The decomposer counts net-new/changed lines. A task producing >100 net-new/changed lines is automatically split. A split task <5 lines merges back into its parent. This bounds every subagent's context to ≤ 100 lines of code-under-test + surrounding context.

Document Index

Document	Path	Description
Architecture Deep Dive	<code>01_architecture/</code>	Detailed layer-by-layer architecture with component diagrams, interface definitions, and contract specifications
SpecKit Extensions	<code>02_extensions/</code>	Custom Spec-Kit skill extensions (<code>speckit-superpowers-bridge-*</code>), manifest definitions, extension installation guide
Helix LLM Integration	<code>03_helix_llm/</code>	Gateway configuration, Brain prompt templates, Knowledge RAG pipeline, Agents orchestration engine, llama.cpp RPC backend setup
TDD & QA Pipeline	<code>04_tdd_qa/</code>	TDD workflow automation, RED → GREEN verdict pipeline, four-layer test coverage, HelixQA Challenge bank integration, quality-gate configuration
Deployment & Operations	<code>05_deployment/</code>	Container setup (podman-compose), MCP server deployment, multi-node cluster provisioning, health monitoring, backup/restore
Risk & Mitigation	<code>06_risk/</code>	Risk register, failure-mode analysis, circuit-breaker design, fallback chains, rollback procedures
Proof-of-Concept	<code>poc/</code>	Working POC walkthrough, sample feature life-cycle trace, benchmark results, known limitations

Quick Start

Prerequisites

Require-ment	Minimum	Verified Command
Git	2.40+	<code>git --version</code>
Podman	4.0+ (rootless)	<code>podman info grep rootless</code>
Python	3.11+	<code>python3 --version</code>
Node.js	20+ (for MCP servers)	<code>node --version</code>
Go	1.22+ (for Helix LLM Gateway / Agents)	<code>go version</code>
llama.cpp	Built with RPC support	<code>llama-server --help grep rpc</code>
Disk space	≥50 GB free (container images + models)	<code>df -h /var/lib/containers</code>

Step – Add the Constitution Submodule

```
# In the consuming project's root (e.g., /mnt/track1/atmosphere-t1):
git submodule add git@github.com:HelixDevelopment/helix_constitution.git constitution
git submodule update --init --recursive
```

Step – Wire CLI-Agent Context Carriers

Add the inheritance pointer to the consuming project's context carriers:

CLAUDE.md (project root):

```
## INHERITED FROM constitution/CLAUDE.md
```

```
> Base agent rules: `constitution/CLAUDE.md` ...
```

AGENTS.md (project root):

```
## INHERITED FROM constitution/AGENTS.md
```

```
> Base agent rules: `constitution/AGENTS.md` ...
```

Do the same for ³ `QWEN.md` , `GEMINI.md` , and any other LLM-specific context carrier.

Step – Install the Bridge Extensions

```
# From the consuming project root:
```

```
bash constitution/scripts/extensions/install_speckit_superpowers_bridge.sh
```

```
# This script:
```

```
# 1. Clones or updates spec-kit, superpowers, superspec, superb as
```

```
# submodules under constitution/submodules/
```

```
# 2. Builds the SuperBridge MCP server (npm install + build)
```

```
# 3. Registers the MCP server in the CLI agent's MCP config
```

```
# 4. Installs the Spec-Kit extensions into .claude/skills/
```

```
# 5. Runs self-check: all extensions parse, MCP server health-check
```

```
# responds, Spec-Kit skills load
```

Step – Run the Self-Validation Suite

```
bash constitution/scripts/extensions/validate_bridge.sh
```

```
# Expected output:
# [PASS] SuperBridge MCP server health-check
# [PASS] speckit-specify extension loads
# [PASS] speckit-plan extension loads
# [PASS] speckit-tasks extension loads
# [PASS] superspec status returns valid JSON
# [PASS] superb check passes on constitution files
# [PASS] Helix LLM Gateway reachable
# [PASS] llama.cpp RPC backend health-check
# _____
# ALL 8/8 CHECKS PASSED — bridge ready
```

Step – Create Your First SpecKit Feature

```
# Operator prompt (any of the six §11.4.140 forms):
# /FEATURE Add multi-channel audio passthrough for Dolby TrueHD
#
# The CLI agent with the bridge wired will:
# 1. speckit-specify → spec.md
# 2. speckit-plan → plan.md
# 3. speckit-tasks → tasks.md
# 4. superb check → constitution gate verification
# 5. superspec status → context snapshot
# 6. superspec tasks → nano-task decomposition
# 7. superspec execute → subagent-driven implementation
# 8. superb critique → independent review
# 9. superb finish → full-suite retest + migration
```

Step – Verify the Autonomous Loop

Check that the feature completed through all layers:

```
cat docs/features/<feature-slug>/Status.md | grep "Validation:"
```

Expected: Validation: PASS (evidence: qa-results/<run-id>/<feature>)

Check the verdict store:

```
ls device/rockchip/rk3588/tests/regression_guard/verdicts/<feature-slug>/
```

Expected: RED_.json + GREEN_*.json*

Check the Issues → Fixed migration:

```
grep "<feature-slug>" docs/Fixed.md
```

Expected: Row with Status "Implemented (→ Fixed.md)"

Key Design Decisions

. Seven-Layer Separation (Why Not Monolithic?)

Decision	Rationale
Governance (L2) separate from Orchestration (L3)	Spec-Kit artifacts are static documents; SuperSpec interprets them. Separating lets Spec-Kit evolve independently and lets SuperSpec adapt to different CLI-agent tool shapes without touching governance files.
Orchestration (L3) separate from Execution (L5)	SuperSpec manages workflows; SuperBridge MCP manages tool contracts. A new skill added to Superpowers updates only the MCP server, not the orchestration logic.
Discipline (L4) is its own layer, not a sub-component of Orchestration	SuperB's enforcement is adversarial — it MUST refuse work that passes orchestration but fails constitutional invariants. Embedding it in SuperSpec would create the self-review blind spot (§11.4.70).
Inference (L6) separate from Execution (L5)	The MCP server calls the Gateway via HTTP; the Gateway abstracts local/remote/cloud routing. Swapping the inference backend (llama.cpp → vLLM → cloud API) changes only Layer 6, nothing above it.

. Nano-Tasks as the Atomic Work Unit (Why ≤ Lines?)

- **Context-bounded subagents** — Each subagent processes ~100 lines of code-under-test + ~200 lines of surrounding context. A 400-line change would need 3 separate subagents; forcing decomposition upfront avoids a subagent silently exceeding its context window.
- **§11.4.197 zero-loss** — A crashed subagent loses ≤100 lines of work. A crashed subagent on a 500-line task loses potentially irrecoverable state.
- **Review granularity** — A 100-line diff gets a thorough adversarial review (§11.4.194). A 1000-line diff gets a rubber stamp.

³

- **Parallelizability** — Nano-tasks with no mutual file dependencies dispatch in parallel (§11.4.103 ≥3 concurrent streams).

. TDD Everywhere (Why Not "Write Tests After"?)

Per §11.4.43, a test authored after the fix is a PASS-bluff — it demonstrates only that the test agrees with the fix, not that the test catches the bug. The bridge enforces:

1. RED test FIRST (`RED_MODE=1` on the pre-fix artifact)
2. RED must FAIL (the defect is reproduced)
3. Implementation lands
4. SAME test flipped to `RED_MODE=0` → GREEN
5. GREEN artifact fingerprint MUST differ from RED (proves the fix deployed)

⁴The SuperB `implementation-gate` refuses to dispatch a nano-task that lacks a pre-existing RED test.

. Local-First Distributed Inference (Why Not Cloud API?)

- **§11.4.126 autonomous-loop** requires continuous execution. A cloud API rate limit (429 Too Many Requests) that kills 5 subagents at once (§11.4.147 forensic anchor) is structurally incompatible with an always-on loop.
- **§11.4.10 credentials-handling** — local inference eliminates the API-key-in-every-request surface.
- **§11.4.6 no-guessing** — a local Gateway logs every prompt + response. A cloud API's opaque logging (or lack thereof) makes forensic debugging of an agent's decision path impossible.
- **Cost** — autonomous loops issue thousands of inference calls per day. At cloud API pricing, that is hundreds of dollars per day; at local inference, it is electricity-only.

. Constitution as the Single Source of Truth (Why Not Per-Project Rules?)

Per §11.4.17 / §11.4.35, universal rules live in the Constitution submodule; project-specific extensions live in the consumer's context carriers. The bridge reads the Constitution file at startup and derives:

- **Allowed status vocabulary** (§11.4.15)
- **Required evidence classes per feature** (§11.4.69)
- **Gating pipeline** (which `CM-*` gates fire at which seam)
- **Propagation targets** (§11.4.17 classification markers)

A new constitutional mandate added to the submodule takes effect in every consuming project on the next `git pull` + `post_update_hook.sh` run (§11.4.164) — no per-project reconfiguration.

Constitutional Mandates Applied

Mandate	How the Bridge Applies It
§11.4 (anti-bluff covenant)	Every gate, review, and verdict carries captured physical evidence (§11.4.5) keyed to a §11.4.69 feature class. No metadata-only PASS, no config-only PASS, no absence-of-error PASS.
§11.4.6 (no-guessing mandate)	Every verdict cites a <code>file:line</code> or a captured-evidence path. The bridge's decomposer, reviewer, and debugger all refuse to emit <code>likely / probably / maybe / seems</code> in any formal output.
§11.4.10 (credentials-handling)	API keys, tokens, and passwords never appear in tracked files. The <code>.env.example</code> template documents required env vars; real values live in git-ignored <code>.env</code> files (<code>chmod 600</code>). The <code>pre-commit</code> hook scans staged files for credential patterns and refuses the commit.
§11.4.65 (universal Markdown export)	Every <code>.md</code> produced by Spec-Kit (<code>spec.md</code> , <code>plan.md</code> , <code>tasks.md</code>) gets synchronized <code>.html + .pdf + .docx</code> siblings via <code>sync_all_markdown_exports.sh</code> , auto-invoked by the <code>post-commit</code> hook (§11.4.75 Layer 4).
§11.4.74 (catalogue-first)	All extensions, skills, MCP servers, and plugins are registered in <code>constitution/actions/registry.yaml</code> via the §11.4.164 <code>post_update_hook.sh</code> . No extension is wired ad-hoc.
§11.4.75 (mechanical enforcement)	Four-layer enforcement: <code>pre-commit</code> hook (refuses orphan <code>.md</code>), <code>commit_all.sh</code> integration (auto-runs exports), <code>pre-push</code> hook (re-checks propagation gates), <code>post-commit</code> hook (auto-repairs drift).
§11.4.76 (containers submodule)	All containerized services (Helix LLM Gateway, SuperBridge MCP, ChromaDB, llama.cpp RPC nodes) are defined in <code>constitution/submodules/containers/compose/</code> and booted via <code>pkg/boot</code> . No ad-hoc <code>podman run</code> .
§11.4.202 (reporting directives)	Every intake path (operator prompt, test failure, review finding, HelixQA Challenge verdict) feeds through <code>report_item.sh</code> , creating a tracked workable item in the SQLite SSoT (§11.4.93) and pushing to the external tracker (§11.4.148 D5).

Mandate	How the Bridge Applies It
§11.4.43 (TDD-fix-discipline)	The five-step TDD workflow (RED → LIVE-ADB-PROBE → GREEN → VERIFY → DOCUMENT) is mechanically enforced by the SuperB <code>implementation-gate</code> + <code>critique</code> + <code>debug</code> + <code>finish</code> gates. No implementation subagent dispatches without a registered RED test.
§11.4.70 (sub-agent-driven default)	Every nano-task dispatches as an isolated subagent. The conductor's role is task decomposition + review; the subagent's role is isolated implementation.
§11.4.116 (real-time sync channel)	The bridge emits <code>conduit.events.jsonl</code> and atomically-updated <code>conduit.status.json</code> so the conductor can tail live progress.
§11.4.147 (crashed-agent respawn)	The Agents component tracks every subagent in the durable registry. Crashed/rate-limited agents are respawned with preserved state + §11.4.84 quiescence check. Agent output with missing verdict = <code>crashed</code> , never <code>complete</code> .
§11.4.161 (rootless container runtime)	All containerized components boot via podman rootless. No <code>sudo docker</code> , no rootful podman.
§11.4.209 (code-review Fable xhigh)	Every code-review subagent dispatches on Fable model at xhigh effort. Opus xhigh is the fallback. Lower-effort or non-Fable/non-Opus review substrate is a refusal.
§11.4.211 (merge-conflict Fable xhigh)	Merge-conflict resolution runs on Fable xhigh (Opus xhigh fallback). Conflict-resolution subagent carries its own scope + evidence.
§11.4.205 (enforced-not-advisory)	Every gate is a commit-wrapper refusal, not prose. Every gate's paired §1.1 mutation FAILs the gate. A gate whose mutation passes is a bluff gate.
§11.4.194 (exhaustive review all scenarios)	The review subagent enumerates the full input/scenario space, proves every "unreachable" assumption, cross-checks against captured runtime evidence.

Mandate	How the Bridge Applies It
§11.4.214 (re-currence-links-not-mints)	Before minting a new item for a test failure, the bridge checks the workable-items DB for an existing item describing the same defect. Match → link + reopen (terminal only).
§11.4.186 (cross-document consistency)	The no-divergence gate runs before export/sync-verify/commit. DEDUP, TIMELINE, CROSS-DOC, INTEGRITY, STRUCTURAL check families.

Directory Structure

```

constitution/docs/research/extensions/speckit_superpowers/implementation/
|
├─ README.md                                ← THIS FILE – master index +
architecture overview
|
├─ 01_architecture/
|   └─ README.md                            Layer-by-layer deep dive with
component diagrams
|   └─ layer_1_workstation.md               Developer workstation specification
|   └─ layer_2_speckit.md                  Spec-Kit governance layer details
|   └─ layer_3_superspec.md                SuperSpec orchestration engine
|   └─ layer_4_superb.md                   SuperB discipline enforcement
|   └─ layer_5_mcp.md                      SuperBridge MCP server contract
|   └─ layer_6_helix_llm.md                Helix LLM inference pipeline
|   └─ layer_7_cluster.md                  Distributed inference cluster
|       └─ contracts/                      Interface contracts between layers
|           └─ spec_to_superspec.md         Spec-Kit → SuperSpec bridge
contract
|       └─ superspec_to_mcp.md              SuperSpec → SuperBridge MCP
contract
|       └─ mcp_to_helix.md                  MCP → Helix LLM contract
|           └─ helix_to_cluster.md          Helix LLM → llama.cpp RPC contract
|
├─ 02_extensions/
|   └─ README.md                            Extensions overview + installation
guide
|   └─ superbridge_mcp/                    SuperBridge MCP server source +
config
|   └─ speckit_bridge_skills/              Extended Spec-Kit skills for bridge
integration
|   └─ skill_manifests/                    Per-skill manifest files
|       └─ constitution_registry.yaml       Action registry entries for bridge
commands
|
├─ 03_helix_llm/
|   └─ README.md                            Helix LLM integration guide
|   └─ gateway/                             Gateway configuration + deployment
|   └─ brain/                               Prompt templates (per skill per
model)
|   └─ knowledge/                          RAG pipeline configuration
|   └─ agents/                             Agent orchestration engine
|       └─ rpc_backend/                     llama.cpp RPC node configuration
|
├─ 04_tdd_qa/
|   └─ README.md                            TDD + QA pipeline guide

```


— red_green_pipeline.md	RED→GREEN verdict pipeline
specification	
— four_layer_coverage.md	Four-layer coverage per §11.4.4(b)
— helixqa_integration.md	HelixQA Challenge bank wiring
— quality_gates.md	Quality-gate configuration +
thresholds	
— 05_deployment/	
— README.md	Deployment guide
— podman_compose/	podman-compose definitions for all
services	
— health_monitoring/	Health-check scripts + dashboards
— backup_restore/	Backup/restore procedures
— cluster_provisioning/	Multi-node cluster setup automation
— 06_risk/	
— README.md	Risk management guide
— risk_register.md	Enumerated risks with severity +
mitigation	
— failure_modes.md	Failure-mode analysis per component
— circuit_breakers.md	Circuit-breaker design + thresholds
— fallback_chains.md	Fallback chains
(local→remote→cloud)	
— rollback_procedures.md	Rollback procedures per scenario
— poc/	
— README.md	POC walkthrough
— feature_trace.md	Sample feature lifecycle trace
(end-to-end)	
— benchmarks.md	Performance benchmarks
— limitations.md	Known limitations + tracked
migration items	

References

#	Title	URL	Relevance
1	Spec-Kit (SDD toolkit)	https://github.com/github/spec-kit	Spec-Kit core: specify, plan, tasks, implement lifecycle
2	Superpowers (agent skills)	https://github.com/obra/superpowers	15+ agent skills: brainstorming, TDD, debugging, code review, refactoring, documentation
3	SuperSpec (SpecKit-Superpowers bridge)	https://github.com/WangX0111/superspec	Bridge orchestration: status, brainstorm, tasks, execute, review
4	SuperB (SpecKit discipline enforcement)	https://speckit-community.github.io/extensions/superb	Discipline gates: check, brainstorm, implementation-gate, critique, debug, finish
5	Helix LLM (distributed inference)	https://github.com/HelixDevelopment/helix_llm	Local-first distributed LLM inference: Gateway, Brain, Knowledge, Agents
6	llama.cpp (inference engine)	https://github.com/ggerganov/llama.cpp	RPC-server mode for distributed inference backend
7	Helix Constitution (governance submodule)	git@github.com:HelixDevelopment/helix_constitution.git	Universal development mandates, mechanical enforcement, agent rules
8	MCP (Model Context Protocol)	https://modelcontextprotocol.io	Protocol for LLM-tool interoperability
9	Podman (rootless containers)	https://podman.io	Rootless container runtime per §11.4.161
10	Claude Code (CLI agent)	https://docs.anthropic.com/en/docs/claude-code	Primary CLI agent implementing the autonomous loop

#	Title	URL	Relevance
11	ATMOSphere (consuming project)	git@github.com:vasic-digital/Android_15.git	Reference consuming project implementing the bridge
12	Constitution submodule documentation	constitution/docs/ (this repository)	Full constitutional mandates, agent rules, and project-independent extensions